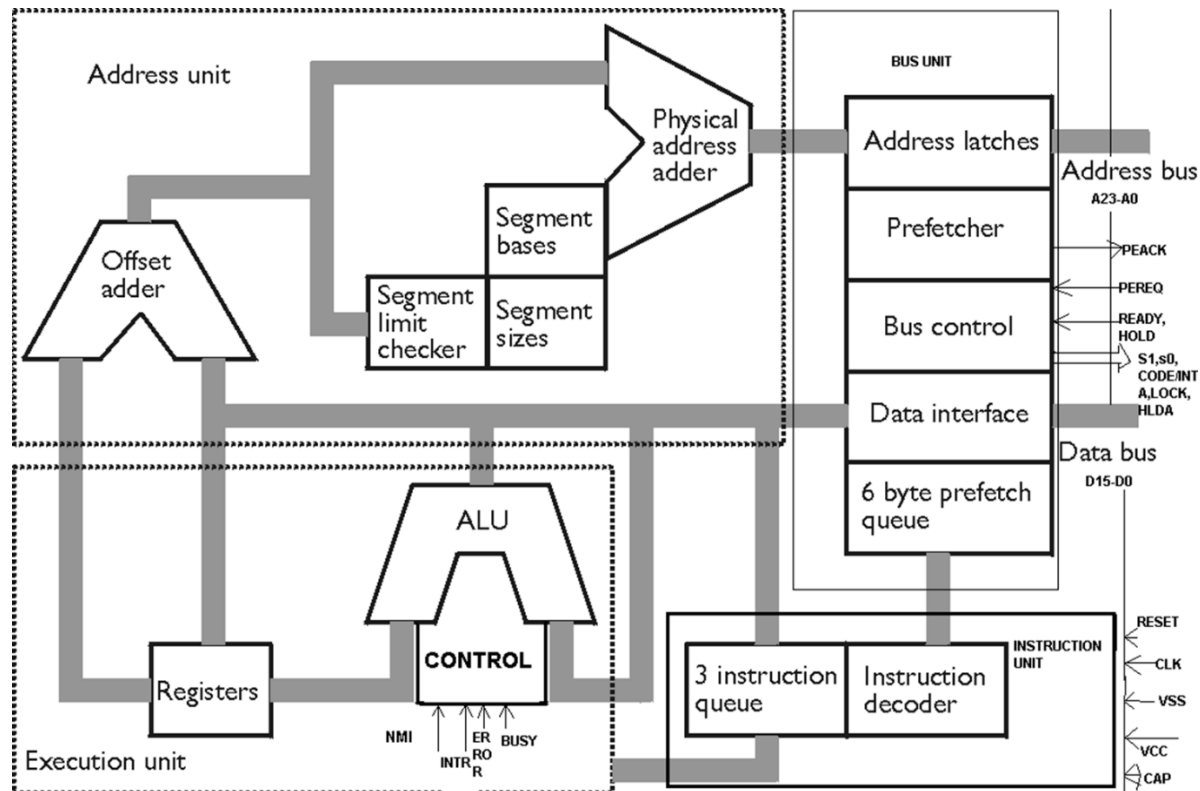


Draw and explain the internal architecture of the 80286 microprocessor. What is the Program Invisible Register?



1. Execution Unit:

This is where actual processing happens.

- **ALU (Arithmetic Logic Unit):**
Performs arithmetic and logical operations.
- **CONTROL (Control Unit):**
Directs how data moves and how instructions are executed.
- **Registers:**
Temporary storage for operands, addresses, and results.
- **Instruction Queue (3-byte):**
Stores instructions fetched from memory before decoding.
- **Instruction Decoder:**
Decodes machine instructions so they can be executed.

2. Address Unit:

Used to calculate the actual physical memory address.

- **Offset Adder:**
Adds the offset (displacement) to base addresses.
 - **Segment Bases and Sizes:**
Used for segmented memory architecture.
 - **Segment Limit Checker:**
Ensures addresses stay within segment limits.
 - **Physical Address Adder:**
Combines base and offset to get the full physical address.
-

3. Bus Unit:

Connects the microprocessor with external devices or memory.

- **Address Latches:**
Hold address lines temporarily during memory access.
- **Prefetcher (6-byte prefetch queue):**
Preloads instructions to speed up execution.
- **Bus Control:**
Manages read/write operations and timing.
- **Data Interface:**
Manages data transfer to/from the processor.

4. Instruction Unit (separate box):

- **Instruction Queue (3 bytes):**
Part of pipelining—holds instructions before decoding.

5. External Connections (Right Side):

- **Address Bus (A23–A0):**
Sends address to memory or I/O.

- **Data Bus (D15–D0):**
Sends or receives actual data.
- **Control Signals:**
RESET, CLK, VSS (ground), VCC (power), etc.

What is the Program Invisible Register?

Program-Invisible Registers are special internal registers used by processors like the 80286, 80386, 80486, and Pentium to manage memory segmentation and protection, but cannot be accessed directly by the programmer—hence the term “*invisible*.”

Key Points:

- These registers store base addresses and limits for critical tables like:
 - GDTR (Global Descriptor Table Register)
 - IDTR (Interrupt Descriptor Table Register)
 - LDTR (Local Descriptor Table Register)
 - TR (Task Register)
- They are automatically loaded when a segment register is loaded.
- Help the CPU in translating logical to physical addresses and in task switching, interrupt handling, and memory protection.

Conclusion:

Program-invisible registers support system-level operations like task management and memory protection, but are not accessible via normal instructions, ensuring secure and controlled operation of the processor.

Explain briefly the 80286 protection mechanism:

80286 Protection Mechanism

The Intel 80286 provides a hardware-based protection mechanism to support multitasking and secure system operation. It uses **segment descriptors** and **privilege levels** to control memory access and protect the operating system.

Key features:

- **Segment limit checking** prevents access beyond allowed memory.
- **Four privilege levels (0–3)** isolate user programs from system software.
- **IOPL (I/O Privilege Level)** controls access to I/O instructions.
- **Access rights byte** specifies segment type and permissions.
- **Task switching** ensures safe execution of multiple tasks.
- **Interrupts** are generated for invalid memory or descriptor access.

This mechanism ensures system stability and prevents unauthorized access to critical resources.

What is the difference between Real Address Mode and Protected Virtual Address Mode in 80286?

80286 Real Address Mode vs Protected Virtual Address Mode (PVAM)

Feature	Real Address Mode	Protected Virtual Address Mode (PVAM)
Memory Addressing	Can access only 1 MB of memory	Can access up to 1 GB of memory
Address Lines Used	Uses A0–A19 (20-bit address bus)	Uses 24 address lines (for 16 MB physical memory), with segmentation + paging logic for 1 GB virtual memory
Start Mode	CPU always starts in this mode after reset	Must be switched manually from real mode
Address Formation	Like 8086: Physical Address = Segment × 16 + Offset	Uses segment selectors and descriptors for address translation
Segment Registers	Contents are used as base addresses directly	Contents act as selectors pointing to descriptors in memory
Interrupt Vector Table	Located in the first 1 KB of memory	Still present but more secure management is possible
System Initialization	Starts from FFFF0H to FFFFFH	Initialization routines change as OS takes over
Segment Size	Max segment size = 64 KB	Same, but more secure and controlled
Memory Protection	 No protection (programs can overwrite each other's memory)	 Supports protection , uses privilege levels to prevent unauthorized access
Swapping	 No swapping	 Swapping allows loading programs from disk to RAM (virtual memory)
Use Case	Simple DOS-like systems	Multi-tasking systems (early Windows, OS/2, etc.)

Explain How protected virtual address mode is used to calculate the physical address

80286 Protected Virtual Address Mode – Physical Address Calculation

In Protected Virtual Address Mode (PVAM), the 80286 uses segment descriptors instead of directly shifting segment addresses like in real mode.

Steps to Calculate Physical Address in PVAM:

1. Logical Address = 16-bit Selector + 16-bit Offset
2. Selector Breakdown (in segment register):
 - 13-bit Index – selects one of 8192 descriptors.
 - 1-bit TI (Table Indicator) – selects table:
 - TI = 0: Global Descriptor Table (GDT)
 - TI = 1: Local Descriptor Table (LDT)
 - 2-bit RPL (Requested Privilege Level) – used for protection, not address calculation.
3. The descriptor table entry (8 bytes) holds:
 - 24-bit base address (starting location of segment)
 - Segment limit
 - Access rights
4. The 80286 fetches the descriptor using the selector index from the selected table (GDT/LDT).
5. Then, it adds the 24-bit base address from the descriptor to the 16-bit offset to compute the 24-bit physical address.

Formula:

Physical Address = Base Address (from descriptor) + Offset

Write short notes on the memory addressing modes of 80286:

Memory Addressing Modes of 80386

The **Intel 80386** microprocessor supports a wide range of **memory addressing modes** that enhance flexibility and efficiency in accessing data and instructions. These modes define how the effective address of an operand is calculated, allowing access to memory in various ways.

◆ **Types of Addressing Modes:**

1. Immediate Addressing

- Operand is part of the instruction.
- Example: MOV AX, 1234H

2. Register Addressing

- Operand is located in a register.
- Example: MOV AX, BX

3. Direct Addressing

- The effective address is directly specified in the instruction.
- Example: MOV AX, [1000H]

4. Register Indirect Addressing

- Address is held in a register like BX, SI, or DI.
- Example: MOV AX, [BX]

5. Based Addressing

- Effective address = Base register + displacement.
- Example: MOV AX, [BX + 04H]

6. Indexed Addressing

- Effective address = Index register + displacement.
- Example: MOV AX, [SI + 04H]

7. Based Indexed Addressing

- Combines base and index registers.

- Example: MOV AX, [BX + SI]

8. Based Indexed with Displacement

- Effective address = Base + Index + Displacement.
- Example: MOV AX, [BX + SI + 08H]

9. Scaled Indexed Addressing (80386-specific)

- Uses a scale factor with index register for efficient array access.
- Example: MOV EAX, [EBX + ESI*4 + 10H]

The 80386's addressing modes provide powerful capabilities for accessing data structures, arrays, stacks, and pointers, making it suitable for both low-level and high-level programming.

Translation of Logical to Physical Address in Intel 80286 :

In **Protected Mode**, 80286 uses a **segmentation mechanism** to translate logical addresses into physical addresses.

1. **Logical Address = Selector + Offset**
2. **Selector points to an entry in:**
 - **Global Descriptor Table (GDT) or**
 - **Local Descriptor Table (LDT)**
(determined by the TI bit in the selector)
3. **The selected Segment Descriptor provides:**
 - **Base address (24 bits)**
 - **Limit (size of segment)**
 - **Access rights**
4. **The Offset is added to the Base Address from the descriptor.**

 **Formula:**

Physical Address = Base Address (from descriptor) + Offset

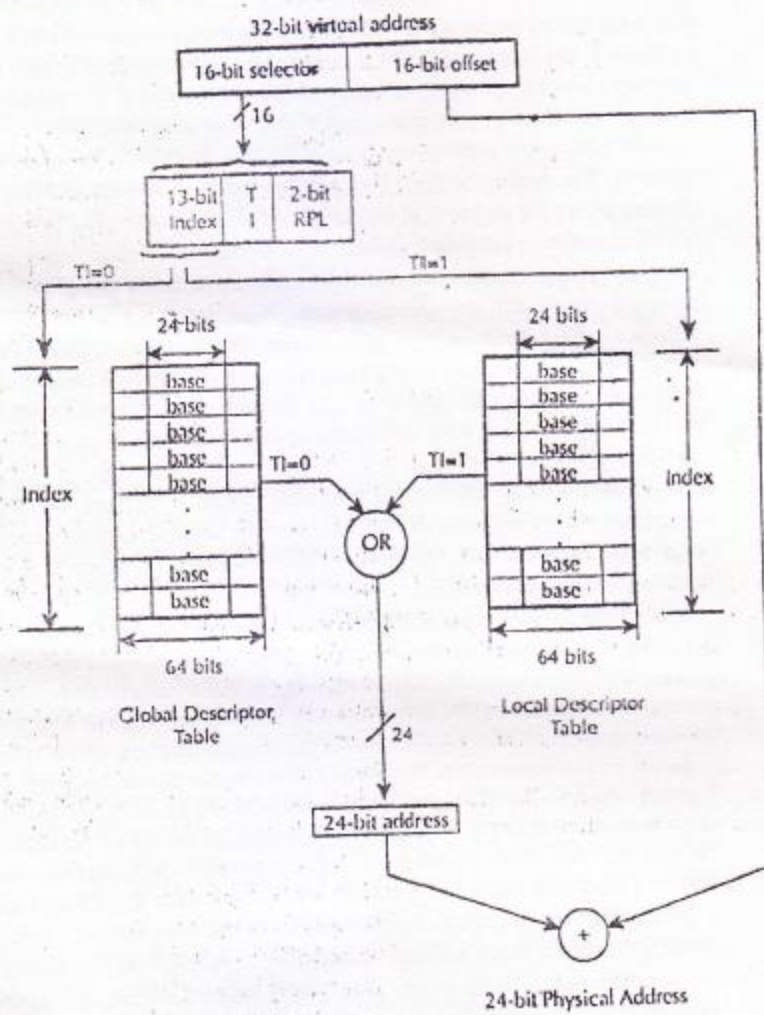


FIGURE 4.3 80286 virtual address translation.

Illustrate how a 64 key keyboard and an 8 digit seven segment display can be connected to an 8279.

Both the keyboard and display are scanned and refreshed under the control of the select signals SL2–SL0. The SL3 pin is not connected because there are only eight display units. Both 3-to-8 decoders have active low outputs. One decoder selects one row of keys while the other decoder enables one of the eight digit drivers.

To demonstrate how to program an 8279, let us assume that the device is connected to a keyboard and multiple-digit display as shown in Fig. 9-35, the 8279's addresses are FFE8 and FFE9, and the interrupt request pin IRQ is not used. First, the device must be initialized by sending a mode set command to the control register. The following instructions set the keyboard/display controller to its encoded keyboard scan mode, with two-key lockout, and its left entry eight 8-bit displays mode:

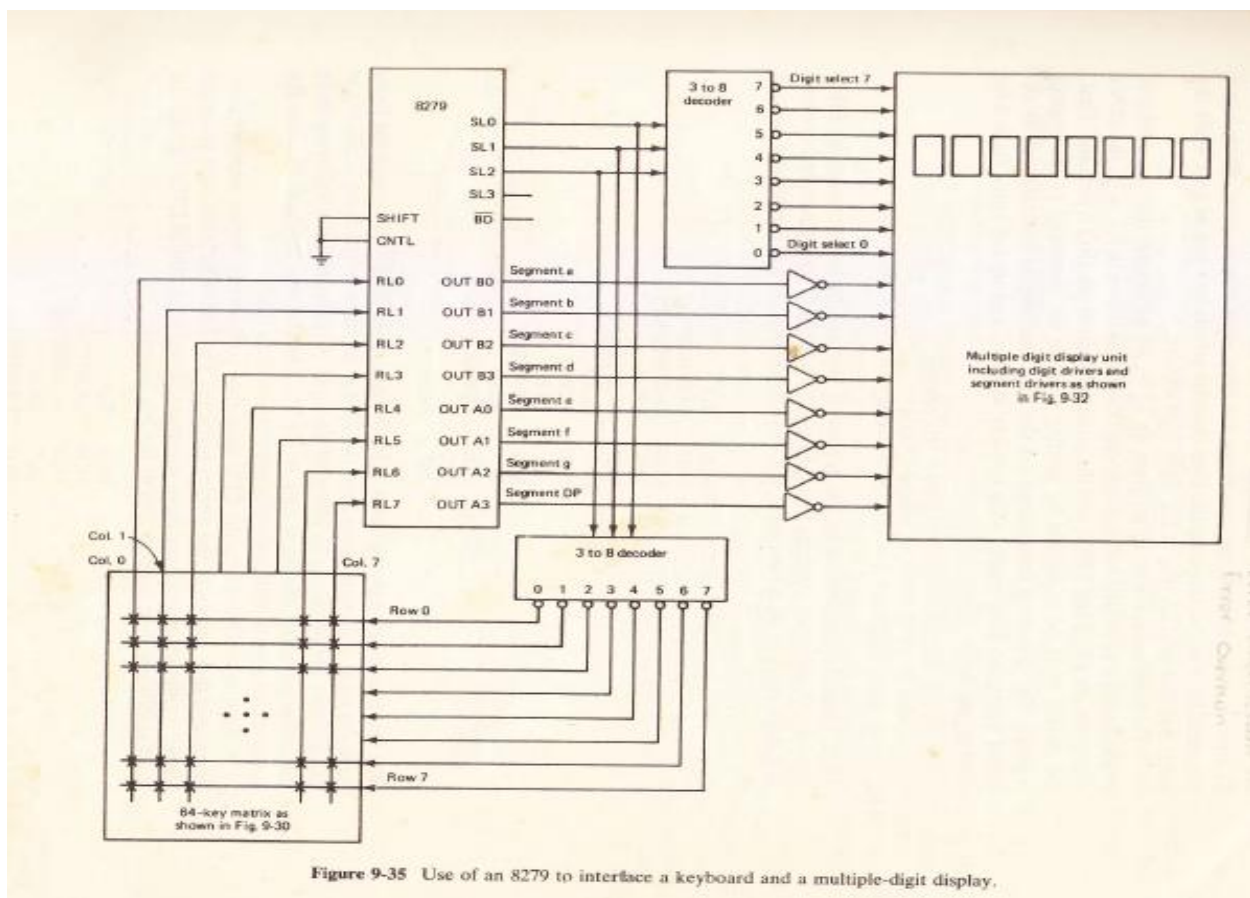


Figure 9-35 Use of an 8279 to interface a keyboard and a multiple-digit display.

Describe briefly synchronous and asynchronous communication

Synchronous Communication

- ✓ A synchronously transmitted character consists of **5 to 8 bits**, optionally followed by a **parity bit**, but **no start or stop bits**.
- ✓ All characters have the **same number of bits (n)**, and transmission time is divided into **groups of n bits** each.
- ✓ The **same clock controls** both the transmitter and receiver, ensuring **synchronization** of data transmission.
- ✓ The transmitter must **send a character in every n-bit interval**. If no character is available, an **underrun** occurs, and the transmitter inserts an **idle character**.
- ✓ **Idle characters** may be interpreted as **errors (underrun errors)** by the receiver, depending on the system.

Asynchronous Communication

- ✓ Data is transmitted **without a shared clock**, and synchronization is achieved using **start and stop bits**.
- ✓ Each character is sent **individually**, typically **5 to 8 bits long**, followed by a **start bit, an optional parity bit, and a stop bit**.
- ✓ The receiver detects the **start bit** to begin reading data and stops after detecting the **stop bit**.
- ✓ It allows **irregular** transmission, meaning characters can be sent **at any time** without waiting for a fixed time interval.
- ✓ Since start and stop bits are added, it has **higher overhead** but is simpler and more flexible.

Key Difference Table

Feature	Synchronous Communication	Asynchronous Communication
Clock	Uses a shared clock	No clock signal, uses start/stop bits
Transmission	Continuous and synchronized	Character-by-character, independent timing
Speed	Faster	Slower
Overhead	Less (no start/stop bits)	More (start/stop bits increase data size)
Examples	USB, Ethernet, SPI	UART, RS-232

Both methods are used depending on the application needs. 🚀